

RESEARCH ARTICLE

Deep Reinforcement Learning Robots for Algorithmic Trading: Considering Stock Market Conditions and U.S. Interest Rates

JI-HEON PARK¹, JAE-HWAN KIM², AND JUN-HO HUH^{2,3}, (Member, IEEE)¹Department of Business Administration, Graduate School of Business, Seoul National University, Seoul 08826, Republic of Korea²Department of Data Science, (National) Korea Maritime and Ocean University, Busan 49112, Republic of Korea³Interdisciplinary Major of Ocean Renewable Energy Engineering, (National) Korea Maritime and Ocean University, Busan 49112, Republic of Korea

Corresponding authors: Jae-Hwan Kim: (jhkim@kmou.ac.kr) and Jun-Ho Huh (72networks@kmou.ac.kr)

ABSTRACT With the development of artificial intelligence, there have been many attempts to incorporate artificial intelligence into algorithmic trading. In particular, reinforcement learning, which aims to solve dynamic decision-making problems, is attracting attention because of its high utilization in algorithmic trading. In this paper, we will implement a simple Deep Reinforcement Learning (DRL) trading robot to check the performance of DRL. In addition, we tried to find out how much performance improvement can be achieved by comparing a robot that learned a single stock data with a robot that learned stock data, market index, and interest rate data. This paper aims to develop a stock investment robot using a Proximal Policy Optimization (PPO) reinforcement learning algorithm and analyze the performance of the robot. The first robot used only the stock data of APPL INC, a single stock, as input, and the second robot used stock data of APPL INC and the S&P 500 index together with US interest rate data as input data. Afterward, the stock investment performance of the two robots for APPL INC was comparatively analyzed using the test data.

INDEX TERMS Machine learning, deep learning, reinforcement learning, artificial intelligence, deep reinforcement learning, quantitative trading, algorithmic trading, robo-advisors, assets under management.

I. INTRODUCTION

Predicting and analyzing financial data has been of interest in the financial world for a long time. Recently, interest in creating a more complete learning model by using more advanced technology in the machine learning (ML) world is increasing. Various methods have been proposed for financial transactions over the past decades. These methods include fundamental analysis [1], technical analysis [2], and algorithmic trading [3]. Many practitioners use a combination of these techniques to transact [4]. Among these, algorithmic trading has gained interest most recently and accounts for about 75% of trading volume on US stock exchanges [3]. The advantages of algorithmic trading, such as computational base, high speed, and risk management without human

emotions, are widespread. One of the key components of these trading systems is predictive power, which can lead to alpha (excess returns), and mathematical and statistical methods are widely used to create models with high predictive power. However, financial data is low in predictive useful information compared to the high volatility and amount of unnecessary noise. The market is also a dynamic and highly volatile place. Therefore, the design of these methods is not straightforward. Most studies on financial data prediction have been focused on regression and decomposition, so the model set up in this way can make appropriate predictions only under specific circumstances.

Creating intelligent agents by automating financial asset transactions has been widely discussed in modern artificial intelligence [5]. In supervised learning, where learning is performed through data with correct answers, there are clear limitations in creating corresponding correct answer data.

The associate editor coordinating the review of this manuscript and approving it for publication was Okyay Kaynak¹.

The lack of labeled data by human experts makes the model's dynamic decision-making difficult. Therefore, a reinforcement learning method in which an agent experiences an unknown environment without such labeled data and simultaneously learns what the right decision is will be able to break through these limitations.

Reinforcement learning has made progress in the following areas so far. Robot navigation [6], Atari gameplay [7], and helicopter control [8]. In addition, in some tests, the performance of reinforcement learning even outperforms human experts in making optimal control decisions [7], [9]. Therefore, these achievements raise questions in the realm of financial asset trading. Can we train reinforcement learning models to beat skilled human traders in financial markets?

Conventional algorithmic trading is more difficult compared to working with reinforcement learning due to the following two problems. The first is the difficulty of summarizing and representing the financial environment. In the field of quantitative financial analysis, extensive research has been conducted on ideal indicators for technical analysis [2]. However, the disadvantage of descriptive analysis is its low generalizability. For example, moving averages explain the trendiness of financial data well, but significant losses can occur in mean-regressive markets [10]. Therefore, the following questions arise: If reinforcement learning is used, can the generalization ability, which is a limitation of existing descriptive analysis tools, be resolved?

The second problem is caused by the operating frequency of system transactions. If you frequently change your trading position (buy, trade), you will incur losses due to trading costs such as fees and slippage. Therefore, when building a non-reinforcement learning system trading robot, the past behavior of the robot along with the current market situation and its corresponding position should be well modeled in the model learning part. This increases the complexity of building a trading robot. So, using reinforcement learning, can we integrate this phenomenon into a trading system without additional complexity?

Deep Reinforcement Learning (DRL), which learns through the trade-off of exploration into unknown territory and exploitation of current knowledge, has been recognized as an advantageous approach to automated financial transactions. The Deep Reinforcement Learning (DRL) framework has two major advantages: portfolio scalability and market model independence [11]. This is because DRL is suitable for solving dynamic decision-making problems (Markov Decision Process) through learning through interaction with an unknown environment. Stock trading in Quantitative Finance is about making dynamic decisions that determine the location, price, and quantity of a trade in a probabilistic and complex stock market. Therefore, DRL is useful to solve the problem of automation of stock trading [12], [13], [14], [15], [16], [17], [18]. DRL trading agents can build multi-factor models by considering various financial factors, which makes it possible to perform algorithmic trading strategies that are

difficult for human traders [19], [20], [21], [22]. Before DRL, reinforcement learning (RL) [23] was applied to complex financial management problems [24], including option pricing, portfolio optimization, and risk management. Moody and Saffell [24] utilized RL to find the optimal policy in stock trading. Deng et al. [25] showed that using deep neural networks can generate more benefits. In addition, the development of more powerful algorithms and investment strategies is making the performance of general machine learning methods and DRL methods more reliable.

For example, DRL implemented sentiment analysis in the analysis of how investors will diversify their assets (portfolio distribution) [22], [26] and strategies for when and how to liquidate their investment positions [27]. This shows the potential of DRLs to provide additional information and insights into investment decisions.

First, in this paper, stock data and S&P 500 index data of APPL INC were collected. In addition, US interest rate data was also collected and used. The collected data consisted of major variables such as open price, highest price, lowest price, closing price, and trading volume, and pre-processing was performed to use it for robot learning and testing.

The first robot was trained through a PPO reinforcement learning algorithm using only stock data from APPL INC as input. The second robot used stock data from APPL INC and the S&P 500 index together with US interest rate data as input. Learning is done repeatedly over a set period, and the robot has improved its ability to learn stock data patterns and make trading decisions based on them.

After the robot's learning was completed, the stock investment performance of the two robots was evaluated using the test data set. Performance evaluation was conducted based on the stock portfolio's return, risk index, and other relevant indicators. Through this, the investment performance of robots for single stocks and robots for multiple stocks was compared and analyzed.

In this paper, the performance of a stock investment robot using the PPO reinforcement learning algorithm was compared and analyzed for single stocks and multiple stocks. The expected result is:

- Since the single stock robot used only APPL INC's stock data, it is expected to be able to focus more on detailed analysis and prediction of stocks.
- The multi-stock robot is expected to be able to reflect the overall trend and economic factors of the market because it considers both APPL INC and S&P 500 index data and US interest rate data.
- Through the performance evaluation results, it is predicted that the difference in stock investment performance of the two robots can be analyzed to derive the strengths and limitations of each robot.

In this paper, the performance of a stock investment robot using PPO reinforcement learning was compared and analyzed for single stocks and multiple stocks. As a result, the pros and cons and limitations of each were identified through

the difference in the performance of the two robots, and through this, the direction for the development of stock investment robots and future research was presented. These results can be used as important guidelines for the development and performance improvement of stock investment robots in the financial market. Figure 1 shows the flow chart of the paper.

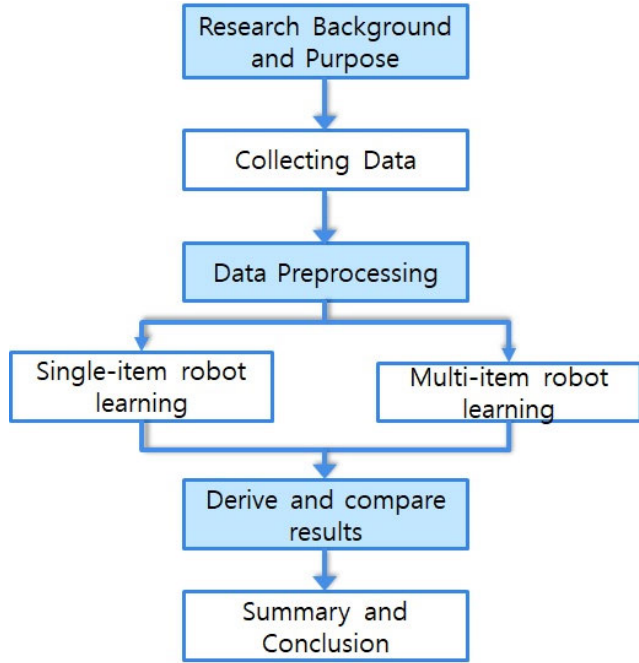


FIGURE 1. Research flow chart.

II. RELATED RESEARCH

A. MARKOV DECISION PROCESS (MDP)

Markov Decision Process (MDP) [28] is a discrete-time stochastic process consisting of state (S), action (A), state transition probability density function (P), reward function (R), and discount factor (γ).

$$MDP = \langle S, A, P, R, \gamma \rangle$$

The MDP problem aims to maximize the expected value of the accumulated reward value. The state transition function is a probability distribution that outputs the probability of going to time $t+1$ when action a is taken from the state at time t . The Reward function outputs the reward value at the next point in time when action a is taken in the state at time t and means the expected value of these reward values. The discount factor γ is a value between 0 and 1 and is a value multiplied by the reward value at the next point to give the future value less than the present value when calculating the sum of the reward values. The sum of the transition function and the reward can be expressed as:

$$P_{ss'}^a = \Pr[S_{t+1} = s' | S_t = s, A_t = a]$$

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

The Bellman equation is needed to find the optimal policy π for maximizing the accumulated reward, which is the MDP problem. This Bellman equation is expressed as a transformation of the Value Function and Action-Value Function. First is the appearance of the value function.

$$v(s) = E[G_t | S_t = s]$$

The Value Function outputs the expected value of the total reward at the current point in time. Next, the Action-Value function can be expressed as follows.

$$q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$$

The action-value function is the expected value of the total reward for taking action a at the current point in time. The Bellman equation can be obtained by transforming the above two functions. The following is the Bellman equation obtained from the value function and the action-value function.

$$v_\pi(s) = E_\pi[R_t + \gamma v_\pi(S_{t+1} | S_t = s)]$$

$$q_\pi(s, a) = E_\pi[R_t + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

By solving these two Bellman equations through the Bayesian theorem, we can express them as follows.

$$v_\pi(s) = \sum_{a \in A} \pi(a | s) [R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v(s')]$$

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a' | s') q_\pi(s', a')$$

π is called policy π . $P_{ss'}^a$ is a state transition function. It can be seen that the factors that determine the value of the Bellman equation include the policy π and the state transition function. π is a probability distribution function, which determines the value of the value function and action-value function by outputting the probability of taking an action at a certain point in time.

$$\pi(a | s) = \Pr[a_t | s_t]$$

$$P_{ss'}^a = \Pr[S_{t+1} = s' | S_t = s, A_t = a]$$

In addition, knowing the state transition function means knowing the future state, but many problems, in reality, are difficult to know the future state. Therefore, the reinforcement learning algorithm uses Model-based RL in an environment where a state transition function can be set, and the Model-free RL algorithm in an environment where a state transition function cannot be set. In the case of model-based reinforcement learning, good performance can be obtained only when an accurate state transition function in the simulation environment is set. In addition, in the case of model-free RL, since there is no state transition function in the simulation environment, a lot of experience is required to find the optimal policy, and it may take a relatively long learning time, and an initial random search step may be required.

As mentioned before, the MDP problem aims to maximize the expected value of the accumulated reward. Therefore, solving the MDP means knowing the optimal value function

that maximizes the expected value of the accumulated reward value.

The optimal value function can be found when we know the probability distribution function π which allows us to maximize the value function. Therefore, reinforcement learning aims to find the optimal policy π that maximizes the expected value of the accumulated reward value.

In other words, finding the optimal policy π through experience is called reinforcement learning.

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

Therefore, the optimal value function is the value function for policy π maximized concerning π , and the optimal state-action function is the state-action function concerning policy π maximized concerning π . In this way, the expected value of the cumulative reward is maximized and the MDP problem can be solved. Figure 2 shows the above process as a flow chart.

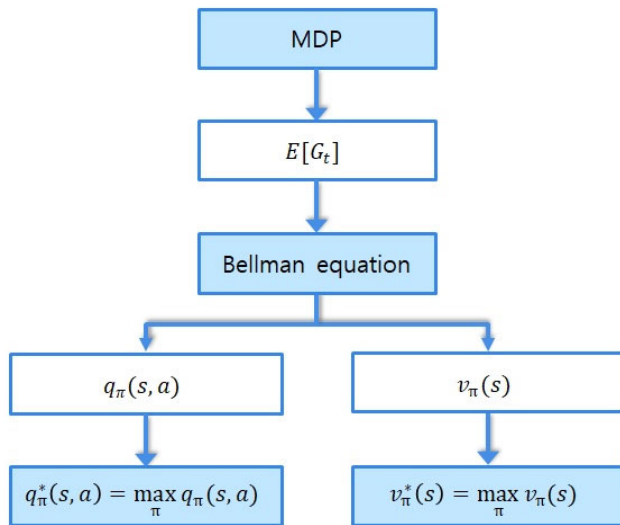


FIGURE 2. MDP flow chart.

B. REINFORCEMENT LEARNING AND DEEP REINFORCEMENT LEARNING

Reinforcement learning is a type of machine learning (ML) in which a machine (agent) learns by interacting with its environment. Reinforcement learning differs from supervised learning, which learns from labeled data that already has an answer. Machines learn through data related to the environment that changes according to their actions (states) and reward data obtained as a result of the machine's actions. Figure 3 is a simple schematic representation of the learning method of reinforcement learning. When action a_t at time t is performed, the environment delivers reward r_t and state s_t along with reward r_{t+1} and state s_{t+1} according to the action at time t .

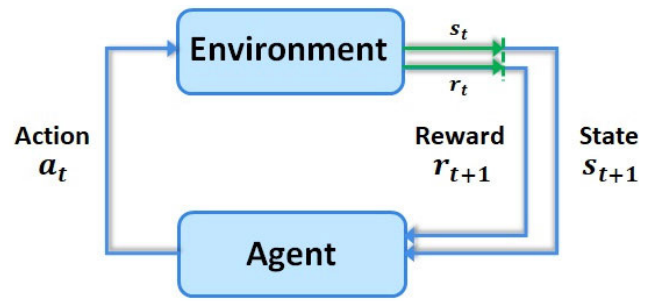


FIGURE 3. A mimetic diagram of the agent's experience in reinforcement learning.

This collected information is used to optimize the policy. At this time, the process of optimizing using an artificial neural network is called deep reinforcement learning. In other words, deep reinforcement learning uses reinforcement learning and artificial neural networks to solve the MDP problem. Information such as action a_t , reward r_t , state s_t is transmitted to the input layer of the artificial neural network. And the artificial neural network generates seek to find the optimal value function $v^*(s)$ or optimal state-action function $q^*(s, a)$ through backpropagation [29]. The detailed optimization process is different for each deep reinforcement learning algorithm. In this project, we chose the PPO algorithm appropriate for our situation. The following paragraphs describe in detail why the PPO algorithm was selected and how the optimization process proceeds.

C. PPO ALGORITHM

PPO (Proximal Policy Optimization) was selected as one of the deep reinforcement learning algorithms because it was suitable for our environment. Because it is difficult to predict the future state of the stock market environment, it is not easy to find the transition probability of the MDP. Therefore, we chose a model-free algorithm in which learning proceeds under the premise that the transition probability is not known [30]. Next, among model-free algorithms, policy optimization algorithms that performed better than Q-learning algorithms for continuous behavior and high-dimensional data were selected [31], [32]. Finally, among the policy optimization algorithms, the PPO algorithm, which showed better performance than other algorithms in terms of implementation, simplicity, and performance, was selected [33].

The PPO algorithm showed better performance than the previous TRPO (Trust region Policy Optimization) algorithm using a clipping technique [33]. Also, this algorithm showed better performance than other policy optimization algorithms through importance sampling [33].

The PPO algorithm is an Actor-Critic algorithm and has an Actor model and a Critic model. Both models use artificial neural networks to find the optimal solution. The actor update uses gradient ascent. The target function of the actor model corresponds to the policy of the MDP problem. In order for the agent to perform the optimal action, the probability distribution policy must be formed so that the probability of

an action that increases the sum of rewards is high and the probability of an action that decreases the sum of rewards is low. Therefore, we use the gradient ascent method to increase the expected total reward. Also, the critical model updates the weights of the artificial neural network in the direction of finding the optimal value function. The objective function is the mean squared error (MSE) of the value expected through the critical model and the actual value calculated by the agent experiencing the value in the state. Therefore, the artificial neural network uses gradient descent to optimize the model.

$$J = E[G_t]$$

In the above equation, the sum of compensation at time t (G_t) is expressed as the addition of the concept of MDP's discount factor. Since solving the MDP problem is to maximize $E[G_t]$, J can be written as:

(For the convenience of the formula, the policy π described above is used as P .)

$$\begin{aligned} J &= E[G_t] \\ &= \int_{s_t} \int_{a_t} \int_{s_{t+1}} \int_{a_{t+1}} \dots G_t P(s_t, a_t, s_{t+1}, a_{t+1}, \dots) \\ &\quad ds_t da_t ds_{t+1} da_{t+1} \dots \\ &= \int_{\tau} G_t P(\tau) d\tau \end{aligned}$$

τ in the above formula is called trajectory in reinforcement learning, and $\tau = s_t, a_t, s_{t+1}, a_{t+1}, \dots$. Therefore, the parameterized policy J_θ is as follows.

$$J_\theta = \int_{\tau} G_t P_\theta(\tau) d\tau$$

In deep reinforcement learning, J_θ is maximized to maximize $E[G_t]$. Therefore, the artificial neural network uses a gradient ascent method to update the parameter θ . Updating θ with respect to the learning rate α is as follows.

$$\theta \leftarrow \theta + \alpha \nabla_\theta J_\theta$$

$\nabla_\theta J_\theta$ is $\int_{\tau} G_t P_\theta(\tau) d\tau$. Logarithm and Bayesian rules are used to extract policy $P(a_t | s_t)$, $P(a_{t+1} | s_{t+1})$, $\dots$ and transition $P(s_{t+1} | s_t, a_t)$, $P(s_{t+2} | s_{t+1}, a_{t+1})$, $\dots$. Since the discount factor γ is a number between 0 and 1, when $t \rightarrow \infty$, $\nabla_\theta J_\theta$ at $t = 0$ can be approximately written as follows.

$$\begin{aligned} \nabla_\theta J_\theta &= \int \sum_{t=0}^{\infty} \left[\nabla_\theta \ln P(a_t | s_t) \sum_{k=t}^{\infty} \gamma^k R_k P_\theta(\tau) d\tau \right] \\ &\simeq \int \sum_{t=0}^{\infty} [\nabla_\theta \ln P(a_t | s_t) G_t] P_\theta(\tau) d\tau \end{aligned}$$

According to the law of large numbers when N is sufficiently large and $X \sim P(x)$, $\frac{1}{N} \sum_{i=1}^N x_i \simeq E[X]$. Therefore, $E[X]$ can be expressed approximately as follows.

$$\begin{aligned} E[X] &= \int_x x P(x) dx = \int_x x \frac{P(x)}{Q(x)} Q(x) dx \\ &= E \left[x \frac{P(x)}{Q(x)} \right], \left(x \frac{P(x)}{Q(x)} \sim Q(x) \right) \end{aligned}$$

$$\simeq \frac{1}{N} \sum_{i=1}^N x_i \frac{P(x_i)}{Q(x_i)}$$

The $\frac{P(x_i)}{Q(x_i)}$ part of the PPO algorithm is called the 'importance weight'. $Q(x)$ means $P(x)$ before θ is updated through the artificial neural network. In this algorithm, $Q(x) = P_{\theta_{old}}(x)$ is used when collecting data (sample) for learning by experiencing the agent.

In addition, to improve algorithm performance, the PPO algorithm uses the Generalized Advantage Estimator (GAE) $\hat{A}_t$ instead of G_t , which are estimate for G_t , using the Advantage function [34]. Therefore, $\nabla_\theta J_\theta$ of the PPO algorithm is converted as follows using the ratio between the previous policy and the updated policy.

$$\nabla_\theta J_\theta \simeq \int_{t=0}^{\infty} \frac{\nabla_\theta \pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t \pi_{\theta_{old}}(a_t | s_t) \pi(s_{t+1} | s_t, a_t) ds_t da_t ds_{t+1}$$

Therefore, maximizing the expected value for the sum of rewards in the PPO algorithm is the same as the right-hand equation above. That is, the surrogate objective function of the PPO algorithm is maximized for the following constraints. The maximization problem of the PPO algorithm is the problem of maximizing the expected value of the product of the ratio of the updated policy to the previous policy and the GAE under the constraint of KL-divergence. That is, as follows.

$$\begin{aligned} \max_{\theta} \hat{E}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t \right] \\ \text{subject to } \hat{E}_t [KL[\pi_{\theta_{old}}(a_t | s_t), \pi_\theta(a_t | s_t)]] \leq \delta \end{aligned}$$

Here, $\pi_\theta(a_t | s_t)$ means the parameterized policy for θ and $\pi_{\theta_{old}}(a_t | s_t)$ means the policy before $\pi_\theta(a_t | s_t)$ is updated. Also, $\hat{A}_t$ means the GAE generated by n samples. At this time, each sample is generated by $\pi_{\theta_{old}}(a_t | s_t)$. And KL is an abbreviation of KL-divergence.

In PPO, this maximization problem was converted into an unconstrained problem through a clipping method, and it showed better performance than the trust region policy optimization (TRPO) algorithm, which was converted into an unconstrained problem using a penalty [34]. The modified surrogate objective function is:

$$\begin{aligned} L_t^{CLIP}(\theta) &= \hat{E}_t [r_t(\theta) \hat{A}_t, clip(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t] \\ \cdot r_t(\theta) &= \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \\ \cdot clip(x) &= \begin{cases} 1 - \epsilon, & x < 1 - \epsilon \\ x, & 1 - \epsilon \leq x < 1 + \epsilon \\ 1 + \epsilon, & 1 + \epsilon \leq x \end{cases} \\ r_t(\theta) \text{ means } &\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \cdot clip(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \end{aligned}$$

outputs $1 - \epsilon$ when $r_t(\theta)$ is less than $1 - \epsilon$, and outputs $1 - \epsilon$ when $r_t(\theta)$ is between $1 - \epsilon$ and $1 + \epsilon$. When $r_t(\theta)$ is greater than $1 + \epsilon$, $1 + \epsilon$ is output. In this way, if $r_t(\theta)$ increases

along the policy gradient and if $\hat{A}_t$ is positive, $r_t(\theta)$ satisfies the constraints of the maximization problem and has a value of $1 + \epsilon$.

Therefore, a constrained maximization problem becomes a problem of maximizing the sum of clipped functions. The sum of the clipped functions in the above formula is maximized through the Gradient Ascent method in the Actor Update process. Therefore, the actor model is updated in the artificial neural network as follows. Here, n means the number of samples to be collected by the agent.

$$\text{Actor update} : \theta \leftarrow \theta + \alpha \nabla_{\theta} \sum_{t=0}^n L_t^{\text{CLIP}}(\theta)$$

When the critical model of the PPO algorithm is trained, the loss value is implemented as the MSE between the value predicted by the model and the value obtained through the agent's experience. Therefore, if the critical model is updated using the gradient descent method to minimize the loss, the following equation is obtained.

$$\text{Critic update} : \omega \leftarrow \omega - \beta \nabla_{\omega} \sum_{t=0}^n [v^{\text{pred}}(s_t) - v^{\text{real}}(s_t)]^2$$

Figure 4 shows the above process as a flow chart. The shaded part in light blue is the part performed by the PPO algorithm. And the part shaded in yellow is the part performed by the Actor model of the PPO algorithm. The orange shaded part is the part processed by the critical model of the PPO algorithm.

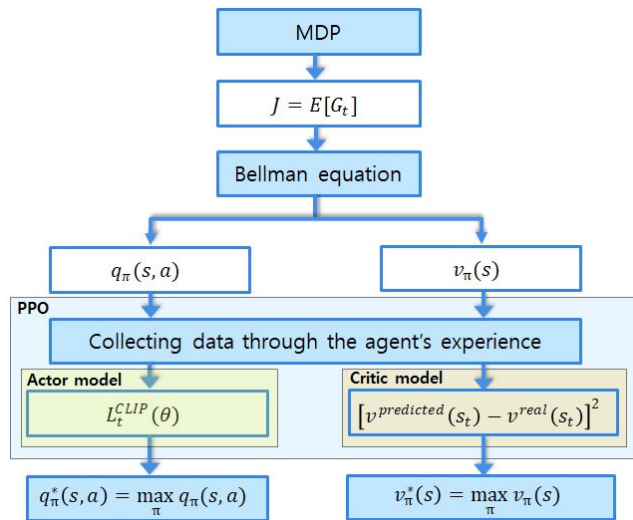


FIGURE 4. PPO algorithm flow chart.

D. COMPONENTS OF REINFORCEMENT LEARNING

Reinforcement learning is a method of solving MDP problems through agent experience. Therefore, the components of reinforcement learning are composed of Observation, Episode, Step, Action function, Reward function, and RL Algorithm. Observation corresponds to the agent's five

senses. You can train an agent by telling it what to observe. Episodes are continued or stopped with the agent's success or failure as a turning point. For example, when training a puppy to give a hand, the puppy corresponds to an agent, and the puppy may take a desirable action in response to a command to ask for a hand, or it may behave incorrectly at all. At this time, the episode does not end when a desirable action is taken, and the episode ends when an incorrect action is performed. Step refers to the unit of action of an agent that occurs within one episode. For example, Step 1 means Agent 1 action. The action function is a function that sets the agent's behavior, and the reward function is a function that determines how much and how to reward the agent for desirable and incorrect behaviors. Finally, the RL Algorithm is an algorithm that learns the agent using the information collected through the agent's experience.

E. CONTRAST BETWEEN FINANCE AND REINFORCEMENT LEARNING TERMS IN ALGORITHM TRADING

The content of reinforcement learning explained above is compared with the terms of financial investment in the system trading domain as shown in Figure 5 below.

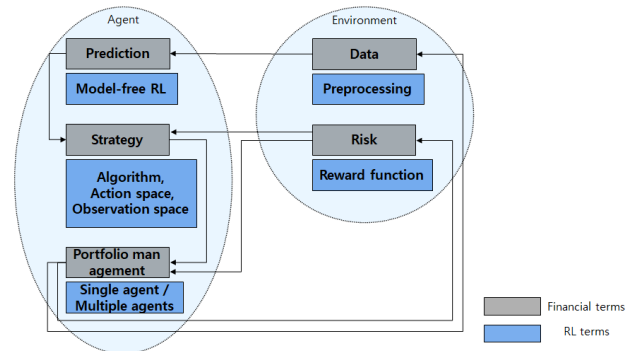


FIGURE 5. Mimetic diagram of terms matching according to the flow of algorithmic trading.

The part corresponding to prediction corresponds to obtaining the transition probability in deep reinforcement learning, and our model applied the model-free reinforcement learning method applied when the transition probability cannot be obtained. The part corresponding to the investment strategy corresponds to the reinforcement learning algorithm, the action function of reinforcement learning, and the observation space. For example, an agent's stock buying, selling, holding, etc. can be designated as an action, and the current price of a stock, stock account balance, etc. can be designated as an observation. The portfolio management part can configure a program according to whether a single agent or multiple agents are used in reinforcement learning. For example, an agent that observes the price of stock A and an agent that observes the price of commodity B can be placed, and the investment portfolio can be managed through the interaction of these two agents. The risk management part can be implemented through the reward function of

reinforcement learning. If the agent's current behavior can cause a loss or potential risk, the reward can be subtracted. Finally, the part corresponding to data can be implemented through pre-processing of learning data.

F. REVIEW OF EXISTING LITERATURE

Algorithmic trading gradually shifted from rule-based trading using technical methods to statistical modeling and time series analysis techniques [35]. ARIMA (Auto Regressive Integrated Moving Average) model and GARCH (Generalized Auto Regressive Conditional Heteroskedasticity) were used as time series analysis techniques [36], [37]. However, Grachev [38] argued that statistical modeling has limitations in that it is difficult to properly respond to dynamic changes in market conditions and to accurately model complex market interactions because it uses a static model.

It has undergone revolutionary changes with advances in machine learning and deep reinforcement learning. A method of learning a trading strategy based on data using a machine learning algorithm and developing a predictive model has received attention.

Meanwhile, Sahu et al. [39] tried to reveal the influence of machine learning on stock market trading by analyzing 66 research papers. In his research, he revealed that Deep Reinforcement Learning (DRL) shows impressive performance compared to other machine learning methods.

Algorithmic trading using deep reinforcement learning that continues to this day is divided into three types depending on the type of algorithm used for learning [40]. These are Critic-only Deep Reinforcement Learning (learning value functions), Actor-only Deep Reinforcement Learning (learning actions), and Actor-Critic Deep Reinforcement Learning (learning value functions and actions simultaneously).

According to Hambly et al. [41], the critic method has been studied the most in this field compared to other methods (actor-only and actor-critic). Because this method is based on the Deep Q-Network [7] algorithm, it focuses on solving the discrete action space problem. However, prices in the stock market are continuous-valued entities. Therefore, there are limitations to applying this method when dealing with multiple stocks. This is because the state space that must be provided to the trading robot and the action space of this robot increase exponentially in multiple stocks [13]. Chen and Gao [17] used Deep Recurrent Q-Network (DRQN) [42], which utilizes Deep Q-Network (DQN) [7] among deep reinforcement learning algorithms. The S&P500 ETF dataset was used as training and testing data. The state data as a market feature given to the agent was the adjusted close price of the S&P500 ETF with a window size of 20 days. Additionally, the actions that this trading robot could take were three discrete actions: buy, sell, and do nothing. The reward the agent received was the difference between the close price at the time the action was taken and the close price the next day. Khan et al. [43] attempted algorithmic

trading using Deep Q-Network (DQN). The data they used for training and testing was historical daily price and volume data for US equities and ETFs. However, unlike Chen and Gao [17], which used only close price as a market feature, 6 features were used (Open, high, low, close, volume, and the day's average price). The trading robot's actions are discrete (buy, sell, hold). Also, the part about the reward function was not dealt with separately.

Actor-only method is a method of directly learning policy. In other words, it is a method of directly learning what action (a_t) to take in state s_t at time t . This method has the advantage of being able to construct a continuous action space. However, learning the optimal policy using this method requires a relatively long time to learn [14]. Deng et al. [25] proposed Fuzzy Deep Direct Reinforcement (FDDR). The reason why their study is meaningful is because, according to the author, it was the first case of using Deep Learning for real-time financial trading at the time. They proved the excellence of their proposed FDDR through stock-index data (stock-IF) and a comparison of trading performance for two commodity markets (silver and sugar). The targets for performance comparison with FDDR were Direct Deep Reinforcement Learning (DDR) without fuzzy representation and Sparse Coding-inspired Optimal Training (SCOT) [44]. The input data given to their FDDR model is the raw price tick. Uncertainty of given raw price ticks is reduced through Deep Learning (DL) along with fuzzy learning concepts [45], [46]. In other words, fuzzy representations [47] are used as part of the data preprocessing step, and then feature learning is performed on the processed data using DL. Then, the features extracted from the trading market are delivered to the Reinforcement Learning (RL) agent, and the agent takes action to maximize the accumulated rewards that can be obtained through trading. At this time, the actions that the agent can take consist of three types of continuous actions: long, short, and hold. Kwak et al. [48] proposed an advanced algorithmic trading model called "Self-attention based deep direct recurrent reinforcement learning with hybrid loss (SA-DDR-HL)." This model focuses on maximizing the reward of the reinforcement learning model by effectively extracting current features from time series stock prices. In this model, the self-attention method helps recognize the temporal dependence of the model's decisions (long, short, or hold a stock). In addition, hybrid learning loss contributes to ensuring that the sequence of hidden features in the input data completely maintains the characteristics of the original state. The states provided to the agent to be learned include market indices (S&P500, NASDAQ100, Dow Jones) and the price (high, low, open, close) and volume of the stock to be predicted. Additionally, what this model predicts is the close price of the stock the next day. There are three actions that an agent can take: short, long, and hold stocks. Finally, the reward is received as the difference between the close price at the time of action and the close price at the previous time. In other words, if the situation is price decrease and short or price increase and long, you will receive a plus reward. And in the

case of price decrease and long or price increase and short, the agent receives minus reward.

Lastly, the actor-critic method is a method of learning the actor model and critic model simultaneously. In other words, this method simultaneously trains an actor model that decides what action (a_t) to take in a given state (s_t) and a critic model that evaluates how valuable the action (a_t) is. As actor-critic algorithms, PPO and A2C have been proven to exhibit superior performance in complex environments [34]. Since our study used the PPO algorithm, it falls under Actor-Critic Deep Reinforcement Learning. Yang et al. [15] introduced an ensemble stock trading strategy using three types of DRL algorithms. These three types of algorithms are all Actor-Critic algorithms: Proximal Policy Optimization (PPO), Advantage Actor Critic (A2C), and Deep Deterministic Policy Gradient (DDPG). Their method showed better performance than a single algorithm or traditional method in terms of Sharpe ratio, a measure of risk-adjusted return. The state given to the Reinforcement Learning (RL) agent is balance, close price of stocks, shares and market indicators. And the action space that the agent can take is continuous. Agents can perform buy, sell, and hold actions, and the amount is considered when buying or selling stocks. Lastly, the function for the reward received by the RL agent focused on maximizing the Sharpe ratio. Majidi et al. [49] attempted to reveal a more effective trading strategy using the TD3 algorithm among the Actor-Critic algorithms and the daily close price of stocks. In particular, they compared an Actor-Critic model that can use a continuous action space and a model that can use a discrete action space. As a result, TD3 showed better performance in terms of profit and sharpe ratio. Because their action space is continuous due to the nature of the algorithm, dynamic trading strategies could be formed. This overcomes the limitations of the discrete model's fixed trading shares. The composition of the action space was long, short, and hold, and each was designed so that "how much" could be considered. In Sate space, in addition to the elements necessary for learning, the change was provided as a percentage using the daily close price of the stock. Lastly, the reward function was composed of the ratio of the initial stock account balance to the balance at the time of investment.

III. DEVELOPMENT OF STOCK INVESTMENT ROBOT

A. DATA COLLECTION AND PREPROCESSING

Data for stock investment robot learning is needed. The collected data is of three types. Among them, Apple stock and S&P500 index were collected at daily intervals. However, in the case of US interest rate data, it is not data collected at intervals of one day. Therefore, for learning, data preprocessing is required to adjust the size of the data and fill in missing values. In addition to this, a process such as type conversion is required to convert the given data to a real number type.

Table 1 is a list of datasets used [54] to train Agent in this paper and a description of each dataset. A total of three datasets were used, all of which were collected through APIs.

TABLE 1. Utilization data list.

Utilization Data	Category	Data Source
Apple stock data (open price, close price, highest price, lowest price, trading volume)	API	-Period : 1999-11-01~ 2023-05-08 -Volume: 8,590 alphavantage.com
S&P500 data (open, close, high, low, trading volume)	API	-Period : 1999-11-01 ~2023-05-08 -Volume: 8,590 - yahoo finance
US interest rate data	API	- Period : 1999-11-01~ 2023-05-08 -Volume: 8,590 - stlouisfed.org

Apple stock data was collected through the API provided by alphavantage.com, S&P 500 data was collected through the API provided by yahoo finance, and US interest rate data was collected through the API provided by stlouisfed.org. In addition, all data was collected at intervals of one day and the length of all data was adjusted based on the relatively short history of Apple stock.

The pre-processing process for each collected dataset was carried out as follows. First, the Apple stock data was indexed after changing the date to datetime64 type, and all values were specified as float64 type.

Finally, among the variables in the dataset, only the open price, highest price, lowest price, closing price, and trading volume were designated as columns. In Figure 6, the appearance of the dataset was printed and confirmed.

The S&P 500, short for Standard & Poor's 500 Stock Index, is one of the most widely known indices in the US stock market. The index tracks and represents the stock prices of 500 publicly traded companies on the US stock market. The S&P 500 includes the largest companies in the US stock market, and the index is calculated by weighting the companies according to their market capitalization. This index is used to determine the health of the US economy and overall trends in the US stock market, and is used as an important benchmark for stock investors to track market performance. The S&P 500 is regarded by many investors and financial experts as a representative indicator of the market, and is widely used to evaluate stock market trends and the performance of investment strategies.

The S&P500 data was preprocessed in the same way as the Apple stock data set. Additionally, slicing was performed to

	Open	High	Low	Close	Volume
Date					
2023-05-08	172.480	173.85	172.11	173.50	55962793.0
2023-05-07	170.975	174.30	170.76	173.57	113453171.0
2023-05-06	170.975	174.30	170.76	173.57	113453171.0
2023-05-05	170.975	174.30	170.76	173.57	113453171.0
2023-05-04	164.890	167.04	164.31	165.79	81235427.0
...	...	...	...	...	...
1999-11-05	84.620	88.37	84.00	88.31	3721500.0
1999-11-04	82.060	85.37	80.62	83.62	3384700.0
1999-11-03	81.620	83.25	81.00	81.50	2932700.0
1999-11-02	78.000	81.69	77.31	80.25	3564600.0
1999-11-01	80.000	80.69	77.37	77.62	2487300.0

```

[8590 rows x 5 columns]
(8590, 5)
Open      float64
High      float64
Low       float64
Close     float64
Volume    float64
dtype: object
datetime64[ns]

```

FIGURE 6. Final dataset for apple stock.

have the same length as the Apple stock data. This is so that our trading robot can take into account the market conditions on the trading day when system trading Apple stock. This idea was conceived by imagining a human stock trader investing in Apple stock, buy, sell, or hold it. In other words, you can use the market conditions at the time of investment as the basis for your actions. Therefore, we wanted to provide data so that our trading robot could consider not only the stock price of the day but also the overall market situation. To do this, it is necessary to match the S&P 500 index, which is different in size, to the same size as the Apple stock, which is the target stock for investment. The difference in the size of this data is because the time when Apple stock was listed and the time when the S&P 500 index was listed do not match. In Figure 7, the appearance of the dataset was printed and confirmed.

	Open	High	Low	Close	Volume
Date					
2023-05-08	4136.98	4142.30	4123.81	4138.12	3.641640e+09
2023-05-07	4084.73	4147.02	4084.73	4136.25	4.186270e+09
2023-05-06	4084.73	4147.02	4084.73	4136.25	4.186270e+09
2023-05-05	4084.73	4147.02	4084.73	4136.25	4.186270e+09
2023-05-04	4082.55	4082.61	4048.28	4061.22	4.920090e+09
...	...	...	...	...	...
1999-11-05	1362.64	1387.48	1362.64	1370.23	1.007300e+09
1999-11-04	1354.93	1369.41	1354.93	1362.64	9.817000e+08
1999-11-03	1347.74	1360.33	1347.74	1354.93	9.144000e+08
1999-11-02	1354.12	1369.32	1346.41	1347.74	9.045000e+08
1999-11-01	1362.93	1367.30	1354.05	1354.12	8.610000e+08

```

[8590 rows x 5 columns]
(8590, 5)
Open      float64
High      float64
Low       float64
Close     float64
Volume    float64
dtype: object
datetime64[ns]

```

FIGURE 7. Final dataset for S&P 500.

Changes in interest rates in the United States can have various effects on stock markets. In general, interest rates and

stock markets tend to have an inverse correlation. This means that when interest rates rise, stock markets are likely to be bearish, and when interest rates fall, stock markets are likely to be bullish.

The main reasons for stock market weakness when interest rates rise are higher costs, returns on alternative investments, lower consumption and investment, and repurchase pressure. When interest rates rise, companies have to pay more to finance them. This causes the company's profits to decrease. In addition, when interest rates rise, the yield on safe assets such as government bonds and bonds rises. This can lead to a tendency for investors to move funds from the stock market to alternative assets that are more profitable than stable. Next, rising interest rates increase the cost of borrowing. As a result, consumers' lending activity may decrease and companies' willingness to invest may decrease. This raises the possibility of a recession and bearishness in the stock market. Finally, rising interest rates could create repurchase pressure in bond markets. Money can flow into the stock market as investors sell bonds in search of higher yields, which can lead to bullish stock markets.

Unlike stocks, US interest rate data is not updated every day. Therefore, there are missing values in the data set collected at daily intervals based on Apple stock. This missing value means that the interest rate renewed on the previous day remains unchanged, so it is set equal to the previously determined interest rate. The rest of the preprocessing process was carried out in the same way as for other datasets. Figure 8 shows the appearance of the preprocessed dataset.

	Value
Date	
2023-05-08	5.15
2023-05-07	5.10
2023-05-06	5.10
2023-05-05	5.10
2023-05-04	5.10
...	...
1999-11-05	4.99
1999-11-04	4.95
1999-11-03	4.97
1999-11-02	4.99
1999-11-01	4.98

```

[8590 rows x 1 columns]
(8590, 1)
Value    float64
dtype: object
datetime64[ns]

```

FIGURE 8. Final dataset on US interest rates.

B. FINAL TRAINING AND TEST DATASETS

In order to check the performance of the model, it is necessary to divide the data to be used for training and the data to be used for testing the performance of the model for the preprocessed datasets. Since our agent's purpose is to invest in future data

from past data, the training dataset is composed of past data and the test data set is composed of future data.

As shown in Figure 9, the training dataset was constructed by slicing all columns from row 1591 to the last row. And as shown in Figure 10, the test dataset was constructed by slicing the dataset from the first row to the 1590th row for all columns.

```
train_appl_df = appl_df.iloc[1591:,:]
train_sp500_df = sp500_df.iloc[1591:,:]
train_us_interest_rate_df = us_interest_rate_df.iloc[1591:,:]
```

FIGURE 9. Final training dataset.

```
test_appl_df = appl_df.iloc[:1591,:]
test_sp500_df = sp500_df.iloc[:1591,:]
test_us_interest_rate_df = us_interest_rate_df.iloc[:1591,:]
```

FIGURE 10. Final test dataset.

C. DEVELOPMENT ENVIRONMENT

We trained the agent using Python and PyTorch. The reinforcement learning algorithm PPO used for learning used the Stable-baselines3 library [51]. The specifications of the PC are as follows. OS: Windows 11 Home 64-bit, Processor: 12th Gen Intel(R) Core (TM) i7-12700H, Memory: 16 GB, DirectX version: DirectX 12, GPU: NVIDIA GeForce RTX 3070 Ti Laptop GPU (VRAM: 8043MB).

D. OBSERVATION, OBSERVATION SPACE

Model 1 and Model 3, which will be shown later, are Agents respectively. The number after Model is named according to the number of datasets used during training.

First of all, Observation is an observation value required for Agent to recognize the current environment situation. Our agent consists of observations as follows. In Model 1, the open price, highest price, lowest price, closing price, and trading volume of APPLE INC are observed values. In Model 3, the open price, highest price, lowest price, closing price, trading volume, and US interest rate of APPLE INC and S&P 500 respectively become observation values. At this time, the number of each data to be observed by the agent at one time is the current time + the data of the previous 5 days.

In addition, the agent needs the following information to make a profit by investing in stocks. Current stock account balance status, Payoff (account balance + number of Apple shares * current price of Apple shares), number of Apple shares held, average purchase price, total number of Apple shares sold, total amount sold. Therefore, this information is added as an observation value in common to both agents. Figure 11 summarizes the above information.

At this time, all observation values are divided by the maximum value of each observation value so that all values exist between 0 and 1. This is to make it possible to see the experimental aspects at a glance during the performance test and to make the agent learn faster.

	Model1	Model3
Observations	• APL INC	• APL INC • S&P500 Index • US Interest Rate
	• Balance • Payoff • Shares • Average purchase price • Total number of stocks sold • Total sales value	

FIGURE 11. Observations of Model 1 and Model 3.

```
def _next_observation(self):
    # Get the data points for the last 5 days and scale to between 0-1
    frame = np.array([
        self.appl_df.iloc[self.current_step: self.current_step + 6, 0].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 1].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 2].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 3].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 4].values / MAX_NUM_STOCKS_APPL,
    ])

    additional_data = np.array([
        self.balance / MAX_ACCOUNT_BALANCE,
        self.payoff / MAX_ACCOUNT_BALANCE,
        self.stocks_held / MAX_NUM_STOCKS,
        self.average_purchase_price / MAX_STOCK_PRICE_APPL,
        self.total_stocks_sold / MAX_NUM_STOCKS,
        self.total_sales_value / (MAX_NUM_STOCKS * MAX_STOCK_PRICE_APPL),
    ])

    # Append additional data and scale each value to between 0-1
    obs = np.append(frame, additional_data, axis=0)
    return obs
```

FIGURE 12. Python code for observations in Model 1.

```
def _next_observation(self):
    # Get the data points for the last 5 days and scale to between 0-1
    frame = np.array([
        self.appl_df.iloc[self.current_step: self.current_step + 6, 0].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 1].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 2].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 3].values / MAX_STOCK_PRICE_APPL,
        self.appl_df.iloc[self.current_step: self.current_step + 6, 4].values / MAX_NUM_STOCKS_APPL,
        self.sp500_df.iloc[self.current_step: self.current_step + 6, 0].values / MAX_STOCK_PRICE_SP500,
        self.sp500_df.iloc[self.current_step: self.current_step + 6, 1].values / MAX_STOCK_PRICE_SP500,
        self.sp500_df.iloc[self.current_step: self.current_step + 6, 2].values / MAX_STOCK_PRICE_SP500,
        self.sp500_df.iloc[self.current_step: self.current_step + 6, 3].values / MAX_STOCK_PRICE_SP500,
        self.sp500_df.iloc[self.current_step: self.current_step + 6, 4].values / MAX_NUM_STOCKS_SP500,
        self.us_interest_rate_df.iloc[self.current_step: self.current_step + 6, 0].values / MAX_INTEREST_RATE,
    ])

    additional_data = np.array([
        self.balance / MAX_ACCOUNT_BALANCE,
        self.payoff / MAX_ACCOUNT_BALANCE,
        self.stocks_held / MAX_NUM_STOCKS,
        self.average_purchase_price / MAX_STOCK_PRICE_APPL,
        self.total_stocks_sold / MAX_NUM_STOCKS,
        self.total_sales_value / (MAX_NUM_STOCKS * MAX_STOCK_PRICE_APPL),
    ])

    # Append additional data and scale each value to between 0-1
    obs = np.append(frame, additional_data, axis=0)
    return obs
```

FIGURE 13. Python code for observations in Model 3.

Figure 12 and Figure 13 shows that the observed values of Model 1 and Model 3 were configured with Python code, respectively.

In the frame np array of Figure 12, model 1 observes APL INC. Numbers 0 to 4 in the np array code mean open, high, low, close, and volume of APL INC data in turn. In the additional_data np array, balance, payoff, shares, average purchase price, total number of stocks hold, and total sales value was entered. Finally, the obs variable was entered by combining these two np arrays, and a function was created

that returns the obs value. As a result, the observation of model 1 was configured as in Figure 11.

In the frame np array of Figure 13, model 3 observes APL INC, S&P500 Index, and US interest rate. Numbers 0 to 4 in the np array code mean open, high, low, close, and volume of APL INC data and S&P500 Index in turn. Finally, we entered us_interest_rate. Next, the additional_data np array consists of balance, payoff, shares, average purchase price, total number of stocks hold, and total sales value, the same as model 1.

Finally, the obs variable was entered by combining these two np arrays, and a function was created that returns the obs value. As a result, the observation of model 3 was configured as in Figure 11.

Next, the observation space must be set according to the size of the observed value. In the case of Model1, the data to be used for learning is 6 by 6 two-dimensional data composed of real numbers as shown in Figure 14.

		6					
APPL Open	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	6
APPL High	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
APPL Low	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
APPL Close	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
APPL Volume	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
Additional data	Balance	Payoff	Stocks held	Average purchase price	Total stocks sold	Total sales value	

FIGURE 14. Observation space of Model 1.

And Model 3 is 2-dimensional data of 12 by 6 as shown in Figure 15. Therefore, the Observation Space is also set accordingly.

		6					
APPL Open	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	12
APPL High	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
APPL Low	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
APPL Close	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
APPL Volume	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
sp500 Open	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
sp500 High	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
sp500 Low	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
sp500 Close	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
sp500 Volume	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
US interest rate	Current	-1 Day	-2 Day	-3 Day	-4 Day	-5 Day	
Additional data	Balance	Payoff	Stocks held	Average purchase price	Total stocks sold	Total sales value	

FIGURE 15. Observation space of Model 3.

Once the agent has created observations, it needs space to hold those observations. This is because accurate observation value information can be provided to the agent. Therefore, in this paragraph, setting the observation space was explained along with the code.

Figure 16 and Figure 17 show the observation space setup Python code. Since all values of the data are set to real numbers, a real number type space must be created.

```
self.observation_space = gym.spaces.Box(
    low=0, high=1, shape=(6,6), dtype=np.float64
)
```

FIGURE 16. Python code for observation space in Model 1.

```
self.observation_space = gym.spaces.Box(
    low=0, high=1, shape=(12,6), dtype=np.float64
)
```

FIGURE 17. Python code for observation space in Model 3.

E. ACTION, ACTION SPACE

Both agents have three actions: buy, sell, and do nothing. At this time, when buying or selling, it was performed at a random rate. Thus, action is continuous, not discrete.

For action a at time t , a one-dimensional np array of x and y values are defined. x is a real number between 0 and 3, and y is a real number between 0 and 1. If x is greater than or equal to 0 and less than 1, 100 $y\%$ of the number of stocks available for purchase with the money currently in the account is purchased. And if x is greater than or equal to 1 and less than 2, 100 $y\%$ of the number of shares currently held is sold. If x is greater than or equal to 2, no action is taken. The action function is shown in the formula below.

$$a_t = (x, y) \begin{cases} \text{Buy } 100 y\%, & 0 \leq x < 1 \\ \text{Sell } 100 y\%, & 1 \leq x < 2 \\ \text{Hold}, & 2 \leq x < 3 \end{cases}$$

$$x \in [0, 3), y \in [0, 1]$$

Meanwhile, Figure 18 shows the implementation of the action space suitable for the above action in Python code.

```
self.action_space = gym.spaces.Box(
    low=np.array([0, 0]), high=np.array([3, 1]), dtype=np.float64
)
```

FIGURE 18. Action space code for Model 1 Model 3.

As mentioned before, the agent has two actions: buying stocks or selling stocks it owns. If you take the action of purchasing and purchase 100 $y\%$ of the purchasable quantity, the balance, which is the account balance containing the money, must be subtracted by the corresponding price. And the number of stocks purchased must be increased by the amount purchased. In addition, it updates observation values necessary for observation. When you make a sale, you do the reverse of when you make a purchase. Figure 19 shows the implementation of the action function in Python code.

The take_action function in Figure 19 is a code implementation of the previously described functions. For the action $a_t = (x, y)$ at time t , action[0] means x and action [1] means y . Therefore, x is assigned to the action type variable and y is assigned to the amount variable. Next, a conditional statement was used to designate buying if x is less than 1 and selling if

```

def _take_action(self, action):
    # Set the current price to a random price within the time step
    current_price = random.uniform(self.appl_df.iloc[self.current_step, 0],
                                   self.appl_df.iloc[self.current_step, 3])
    action_type = action[0]
    amount = action[1]
    if action_type < 1:
        # Buy amount % of balance in stock account.
        total_possible_num = self.balance / current_price
        stocks_bought = total_possible_num * amount
        prev_cost = self.average_purchase_price * self.stocks_held
        additional_cost = stocks_bought * current_price
        self.balance -= additional_cost
        self.average_purchase_price = (prev_cost + additional_cost) / (self.stocks_held + stocks_bought)
        self.stocks_held += stocks_bought
    elif action_type >= 1 and action_type < 2:
        # Sell amount % of stocks held
        stocks_sold = self.stocks_held * amount
        self.balance += stocks_sold * current_price
        self.stocks_held -= stocks_sold
        self.total_stocks_sold += stocks_sold
        self.total_sales_value += stocks_sold * current_price

```

FIGURE 19. Action function code of Model 1 and Model 3.

x is greater than or equal to 1 and less than 2. If x is greater than or equal to 2, no action is taken.

F. REWARD FUNCTION

When the profit is greater than or equal to 0 or when the profit is less than 0, the reward is + by the profit, and -opportunity cost when the current account balance and the initial account balance are the same and the current step is 365. This is because it is thought that giving rewards equal to the amount of profit will help to distinguish which actions are good and which actions are bad. Opportunity cost is calculated when the current account balance and the initial account balance are the same, and the investment period has passed 365 days, and the reward equal to the opportunity cost is -. By doing this, we tried to prevent the Agent from doing nothing for a year and leaving the profit at zero. The opportunity cost calculation we used is different from the existing opportunity cost calculation. Opportunity cost is calculated by dividing the current step by the maximum step multiplied by the current account balance. This is to let the Agent know that there is a reward if it continues to invest without failing.

When the reward value becomes negative, Agent ends the current episode. Then, the environment is initialized and Agent tries to invest in stock again to increase the reward. The Reward function is expressed as a formula below.

$$\text{Reward} = \begin{cases} +\text{profit} \\ -\text{OC} \end{cases}$$

$\cdot \text{profit} = \text{CAB} + \text{shares} \times \text{CSP} - \text{IAB}$
 $\cdot \text{OC} = \text{Opportunity Cost}$
 $\cdot \text{CAB} = \text{Current Account Balance}$
 $\cdot \text{IAB} = \text{Initial Account Balance}$
 $\cdot \text{shares} = \text{number of stocks held}$
 $\cdot \text{CSP} = \text{Current Stock Price}$
 $\cdot \text{CS} = \text{Current Step}$

```

reward = profit
profit = self.payoff - INITIAL_ACCOUNT_BALANCE
self.payoff = self.balance + self.stocks_held * current_price
if profit == 0 & self.current_step > 365:
    reward = -opportunity_cost
opportunity_cost = (self.current_step / MAX_STEPS) * self.balance

```

FIGURE 20. Code of reward function.

Meanwhile, Figure 20 shows the reward function written in code. Profit was defined as the reward variable. In this way, the agent knows that the action is wrong if the profit is negative, and the action is good if the profit is positive. The other codes are those that set the variables according to the reward function formula above.

G. EPISODE AND STEP

In order to solve the MDP problem through reinforcement learning, the agent's experience is required. In other words, reinforcement learning aims to find the optimal policy π that maximizes the "expected value of the accumulated reward value" G_t through the data accumulated through experience.

Figure 21 shows that Agent collects data while gaining experience. If the agent takes an action that seems to increase G_t in the current situation according to policy π , the environment outputs the reward for that action, the next observation value, and the Done value, which is a Boolean value indicating whether the step is finished or not. If done is true, the environment is reset to the initial environment and the Agent tries again from the beginning to increase G_t . Conversely, if Done is False, the Agent proceeds with the action without initializing the environment.

Here, the environment is initialized means that the balance (stock account balance) is initialized to \$10,000 in our project, and various variables related to it are initialized to initial values. The data accumulated through the agent's

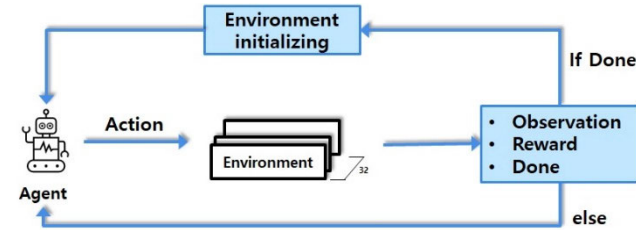


FIGURE 21. A schematic diagram of the agent's experience.

experience is then used to improve the policy π , which is performed through a reinforcement learning algorithm. In this case, if this algorithm uses NN, it is called deep reinforcement learning.

Figure 22 is the code that implements this experience ending and restarting (Episode ending and restarting). As explained earlier, the reward the agent receives is profit. If Profit is greater than 0, the episode does not end as a good action. Conversely, if profit is negative, the episode must end as a wrong action. This is implemented using conditional statements in Python code.

```

profit = self.payoff - INITIAL_ACCOUNT_BALANCE
if profit >= 0:
    reward = profit
    self.current_episode_reward += reward
    if self.st_counter == 0:
        self.cumulative_rewards.append(reward)
        self.st_counter += 1
    else:
        self.cumulative_rewards.append(reward + self.cumulative_rewards[self.st_counter-1])
        self.st_counter += 1
    done = False
else:
    reward = profit
    done = True
    if self.st_counter == 0:
        self.cumulative_rewards.append(reward)
        self.st_counter += 1
    else:
        self.cumulative_rewards.append(reward + self.cumulative_rewards[self.st_counter-1])
        self.st_counter += 1
    if self.ep_counter == 0:
        self.episode_rewards.append(reward)
        self.ep_counter += 1
    else:
        self.episode_rewards.append(self.current_episode_reward + self.episode_rewards[self.ep_counter-1])
        self.ep_counter += 1
  
```

FIGURE 22. Code that implements ending or advancing an agent's episode.

H. HYPERPARAMETERS OF PPO ALGORITHM

In PPO, there are various hyperparameters. These hyperparameters are values for implementing the formula of the PPO algorithm described above. We used stablebaselne3 [51], which provides the PPO algorithm as a library. Table 2 summarizes the hyperparameter values changed for our project from the corresponding library and the major hyperparameter values in the PPO algorithm [33]. Since it was confirmed through subsequent experiments that learning proceeded smoothly, the corresponding values were used without further adjustment.

In this paragraph, we explain the meaning of each hyperparameter in Table 2 and how the corresponding values are set. Policy means the type of artificial neural network of the model. Here, an artificial neural network structure consisting of several hidden layers was selected using Multi-Layer Perceptron (MLP). Discount factor means the gamma value of MDP. When calculating the expected value of the total

TABLE 2. PPO algorithm hyperparameters.

Hyper Parameter	Value
Policy	Multi-Layer Perceptron
Discount factor	0.99
GAE parameter	0.95
Horizon(T)	64
Epsilon	0.2
Learning rate	3.0e-4
Number of epochs	16
Batch size	128
Buffer size	2048
Number of actors	32
Entropy regularization	0
Total max steps	1Million, 1Million
Normalize advantage	True
Value function coefficient	0.5
Max gradient norm	0.5

compensation value from the current point, this value must be set to a value between 0 and 1 in order to discount future compensation compared to the current compensation value. As in the PPO algorithm paper, we set a value of 0.99 [33]. The GAE parameter is a hyperparameter required when applying the generalized advantage estimator. This value was also set to 0.95, like the author of the PPO algorithm paper [33]. Horizon(T) is the number of samples to be collected per environment, also called trajectory. Our agent collects 64 samples from each of 32 environments in parallel. Therefore, this value was set to 64. Epsilon is a ratio multiplied by the degree of exploration so that the degree of exploration can be reduced each time the episode is repeated when the agent explores. This value was also set to 0.2, like the author of the PPO algorithm. Learning rate means learning rate of artificial neural network of actor model and critic model. In our case, the default value of 0.0003 of stablebaselne3 [51] was used. The number of epochs is a number that determines how many times our two artificial neural networks (Actor model & Critic model) will be repeatedly trained from the input dataset. Also, the number of epochs was set to 16 for the following reasons. The number of environments is 32 and the number of samples to be collected by each environment is 64, so the total number of samples to be stored in the buffer is 2048(32 × 64). Then, 128 samples, the batch size, are randomly extracted from this buffer which is called experience replay method. The 128 randomly extracted samples are used to learn the actor model and critic model. So, 2048 divided by 128 is 16. That is, the number of epochs is 16. Next, entropy regularization is a value that determines the degree of exploration. As entropy increases, the variance of the policy, which is the probability

distribution that determines the action, increases. Accordingly, the agent performs various actions, which increases the degree of exploration. By setting the value to 0, the degree of additional experience was not set. The total max steps are set to 1 million for model 1, which is trained using apple stock data, and model 3, which is trained using apple stock data, S&P 500 index, and interest rate data. This was set through experiments to prevent overfitting and underfitting. Normalize advantage determines whether or not to normalize for GAE. It was set to True for the stability of learning. The value function coefficient is a constant for the loss value calculation of the value function calculated by the critic model. In other words, $[v^{pred}(s_t) - v^{real}(s_t)]^2$ is the value that is multiplied when calculated. In our case, we used the default value of 0.5 provided by stablebaseline3 [51]. Lastly, Max gradient norm means gradient clipping of the PPO algorithm. Gradient clipping prevents large changes during policy updates and restricts policy updates so that the new policy does not deviate significantly from the old policy. By setting this value, the learning speed can be improved [52]. We used the default value of 0.5 for stablebaselines3 [51].

I. OVERALL LEARNING PROCESS DESCRIPTION

Our training datasets are APPL INC, s&p500 index, and interest rate datasets with sizes of 8590×5 , 8590×5 , and 8590×1 , respectively, of which 7000 were used as training datasets. In the case of Model 1, only APP INC data was used, and in Model 3, APPLINC, s&p500 index, and interest rate data sets were used. In addition, in order to increase the learning speed, 32 identical environments were created to learn in parallel.

First, we initialize θ , which is a policy parameter that determines the agent's behavior and is a parameter of the Actor model, and ω , which is a parameter of the Critic model. And while interacting with 32 environments in parallel, the agent collects 64 samples of observation values for each environment, calculates the GAE parameters for each of these collected samples, and stores them in a buffer of size 2048.

At this time, the action a_t taken by the Agent at time t is selected as the one with the highest probability $P(a_t)$ for each action output by $\pi_{\theta_{old}}$, which is the previous policy. Also, $v(s_t)$, the value at time t , is calculated by the previous value function, $Q_{\omega_{old}}$. This value function is optimized by the critic model. After that, sample t , the sample at time t , is composed of s_t , a_t , $P(a_t)$, $v(s_t)$, and s_{t+1} . Then, GAE $\hat{A}_t$ is obtained from sample t of each environment.

Then, samples are extracted from the buffer as many as 128, which is the batch size, according to the experience replay method [53].

$\pi_{\theta_{old}}$ and $Q_{\omega_{old}}$ output a new behavior a and a new state value $v(s_t)$ from 128 randomly selected samples. The new a and new $v(s_t)$ output in this way is used to update the parameter θ of the Actor model and the parameter ω of the Critic model. This iteration is repeated for 16 epochs. After

this iteration is over and the weights of the actor model and the critic model are updated, it returns to the sample collection stage.

Back in the collection phase, before the agent acts according to the policy, $\pi_{\theta_{old}}$, which determined the agent's action in the previous iteration, is updated with the learned $\pi_{\theta_{new}}$. And $Q_{\omega_{old}}$, which outputs the value of the state, is updated with the learned $Q_{\omega_{new}}$. This series of sample collection process, Actor model and Critic model update process, and policy update are repeated as many as the set total max steps divided by the buffer size. Figure 23 shows these processes in a schematic diagram. Meanwhile, in Algorithm 1, this process is represented by pseudo code. Lines 8 and 9 are the process of updating the weights of the model. When actor update and critic update occur, gradient ascent method is used for actor update and gradient descent method is used for critic update.

J. ACTOR MODEL

This section explained how the actor model in the PPO algorithm was constructed in our project. Figure 24 is the appearance of the actor model. Observation values are input to the input layer of the actor model.

And in the output layer, the probabilities for the three actions are output by the Softmax activation function [54]. Each probability is for the agent buying the number of stocks, selling the number of stocks, and holding the term. As shown in the figure, the hidden layer consists of four layers. And the number of nodes in each layer is configured as set in the previous section.

Algorithm 1 When PPO Is Applied to Our Environment

```

1: Input: Initialize policy parameters  $\theta$  and value function parameters  $\omega$ 
2: for iteration = 1, 2, ...,  $\frac{\text{total steps}}{\text{buffer size}}$ 
3:   for actor = 1, 2, ..., 32 do
4:     Run  $\pi_{\theta_{old}}$  for the number of 2048 timesteps
5:     Collect num of 2048 samples
6:     Compute GAE  $\hat{A}_0, \dots, \hat{A}_{2048}$ 
7:     for number of epochs = 16 do
8:       Actor update:  $\theta \leftarrow \theta + \alpha \nabla_{\theta} \sum_{i=0}^{128} L_t^{CLIP}$ 
9:       Critic update:
          $\omega \leftarrow \omega - \beta \nabla_{\omega} \sum_{i=0}^{128} (v^{predicted}(s_t) - v^{real}(s_t))^2$ 
10:    end for
11:  end for
12:   $\theta_{old} \leftarrow \theta, \omega_{old} \leftarrow \omega$ 
13: end for
```

K. CRITIC MODEL

This section explained how the critic model in the PPO algorithm was constructed in our project. Figure 25 shows the appearance of the Critic model. When an observation value enters the input layer, the value for that state is output. The meaning of value in MDP is determined by the degree of

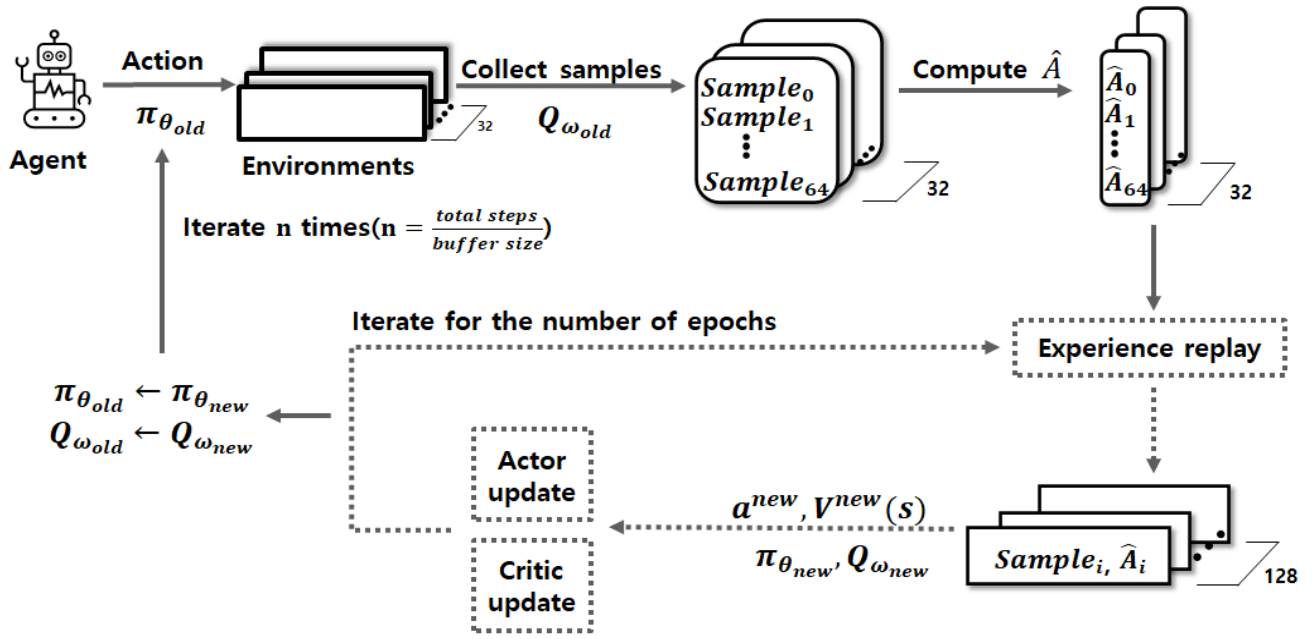


FIGURE 23. Agent's overall learning process.

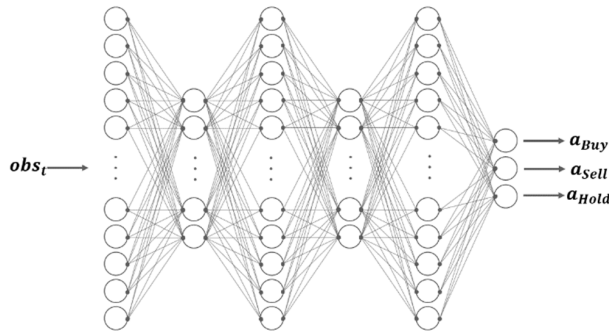


FIGURE 24. Actor model artificial neural network.

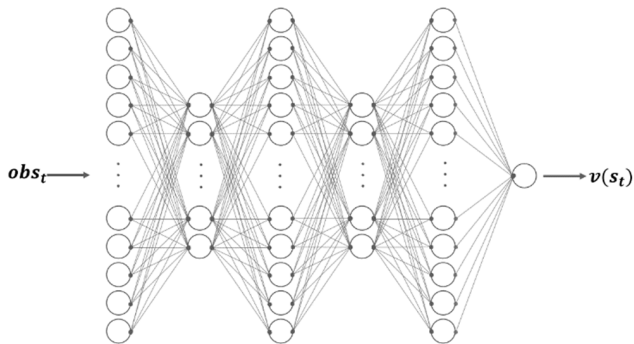


FIGURE 25. Critic model artificial neural network.

overall reward that can be received in the future. As shown in the figure, the hidden layer consists of four layers. And the number of nodes in each layer is configured as set in the previous section.

Activation Functions

Sigmoid
 $\sigma(x) = \frac{1}{1+e^{-x}}$

tanh
 $\tanh(x)$

ReLU
 $\max(0, x)$

ELU
 $\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$

FIGURE 26. A representative activation function of an artificial neural network.

L. MODEL ARCHITECTURE: ACTIVATION FUNCTION

There are various types of activation functions that can be used in artificial neural networks. In this project, the tanh activation function was used.

Figure 26 shows an activation function often used in artificial neural networks. Among them, the tanh activation function, unlike other functions, has the steepest form at the beginning, and therefore, the results output by the activation function are more diverse than other activation functions. In reinforcement learning, it is important to have various experiences in the beginning and not miss important experiences, so it was judged that it would be desirable to use an activation function with a large difference in output, such as tanh, for this purpose.

M. MODEL ARCHITECTURE: MODEL 1

In the case of Model 1, the training data is 7000 by 5. Also, the observation space is 6 by 6. Therefore, the size of the neural network was set considering that it is rather small training data. As a result of checking overfitting and underfitting and

```

env_1.reset()
model_1 = PPO("MlpPolicy",
              env_1,
              n_steps=64,
              batch_size=128,
              n_epochs=16,
              verbose=1,
              policy_kwargs=dict(net_arch=dict(pi=[36,72,36,72], vf=[36,72,36,72])))
model_1.learn(total_timesteps=1000000,callback=callback_1)

```

FIGURE 27. Model architecture python code of Model 1.

setting them appropriately, as above, it was desirable to set the hidden layer to four layers, and the number of nodes to be set to 36, 72, 36, and 72, respectively. And the performance was the best when the total number of steps was 1 million. Figure 27 shows the application of these elements to the PPO library.

Other variables are variables that need to be set when providing the PPO algorithm in stablebaseline3 [40]. Briefly, “MlpPolicy” means the policy to be used by the model, and “env_1” means the environment where the model will be trained. And “n_steps, batch_size, and n_epochs” were set as in Table 2. “Verbose” is whether or not to output information about this model, and we set it to 1 to confirm the information. Other parameters are the same as those set in Table 2, which are the same as the default values of the PPO algorithm provided in stablebaseline3.

In Figure 28, the structure of the neural network for Model1 is output. Tanh was used as the activation function, and it can be confirmed that as many hidden layers as the set number of nodes are configured. Policy_net refers to the Actor model in the PPO algorithm, while value_net represents the Critic model. “in_features” is the number of nodes that receive input values, and “out_features” is the number of nodes that output values. If you look at the number of nodes, you can see that it is configured as set in Figure 27.

```

(mlp_extractor): MlpExtractor(
  (policy_net): Sequential(
    (0): Linear(in_features=36, out_features=36, bias=True)
    (1): Tanh()
    (2): Linear(in_features=36, out_features=72, bias=True)
    (3): Tanh()
    (4): Linear(in_features=72, out_features=36, bias=True)
    (5): Tanh()
    (6): Linear(in_features=36, out_features=72, bias=True)
    (7): Tanh()
  )
  (value_net): Sequential(
    (0): Linear(in_features=36, out_features=36, bias=True)
    (1): Tanh()
    (2): Linear(in_features=36, out_features=72, bias=True)
    (3): Tanh()
    (4): Linear(in_features=72, out_features=36, bias=True)
    (5): Tanh()
    (6): Linear(in_features=36, out_features=72, bias=True)
    (7): Tanh()
  )
)

```

FIGURE 28. Structure of Model 1's neural network.

N. MODEL ARCHITECTURE: MODEL 2

In model 3, the size of the neural network was set in the same way. However, in the case of Mode 3, the learning data size is 7000 by 11 and the observation space is 12 by 6, so it is relatively larger than Model 1. Accordingly, the size of the neural network was increased accordingly. The number of hidden layers is the same as 4 layers, but the number of nodes in each layer is composed of 72, 144, 72, and 144 nodes. It was confirmed that the best performance was obtained when the total number of steps was 1 million through experiments. Figure 29 shows the application of these elements to the PPO library. The rest of the settings are the same as described in Figure 27.

```

model_3 = PPO("MlpPolicy",
              env_3,
              n_steps=64,
              batch_size=128,
              n_epochs=16,
              verbose=1,
              policy_kwargs=dict(net_arch=dict(pi=[72,144,72,144], vf=[72,144,72,144])))
model_3.learn(total_timesteps=1000000,callback=callback_3)

```

FIGURE 29. Model architecture python code of Model 3.

```

(mlp_extractor): MlpExtractor(
  (policy_net): Sequential(
    (0): Linear(in_features=72, out_features=72, bias=True)
    (1): Tanh()
    (2): Linear(in_features=72, out_features=144, bias=True)
    (3): Tanh()
    (4): Linear(in_features=144, out_features=72, bias=True)
    (5): Tanh()
    (6): Linear(in_features=72, out_features=144, bias=True)
    (7): Tanh()
  )
  (value_net): Sequential(
    (0): Linear(in_features=72, out_features=72, bias=True)
    (1): Tanh()
    (2): Linear(in_features=72, out_features=144, bias=True)
    (3): Tanh()
    (4): Linear(in_features=144, out_features=72, bias=True)
    (5): Tanh()
    (6): Linear(in_features=72, out_features=144, bias=True)
    (7): Tanh()
  )
)

```

FIGURE 30. Structure of Model 3's neural network.

In Figure 30, the structure of the neural network for Model3 is output. Tanh was used as the activation function, and it can be confirmed that as many hidden layers as the set number of nodes are configured. The rest of the settings are the same as described in Figure 28.

IV. PERFORMANCE EVALUATION

Before starting this paragraph, the dataset used [54] is attached. Attached to this link is the original dataset for:

1. Dataset used to train and test the model.
2. Reward dataset for each episode of model 1 and model 3 for training data.
3. Reward dataset by episode for test data.
4. Reward data set collected by repeating the experiment 1000 times for the two models investing as much as 100 episodes for the test data.



FIGURE 31. Learning performance on training data.

A. PERFORMANCE COMPARISON AND ANALYSIS OF THE TWO ROBOTS ON TRAINING DATA

In this section, we checked the reward table according to the episode to check whether the learning of the two robots proceeded well with respect to the training data.

Figure 31 shows the performance of the two models on the training data. The x-axis is the episode, and the y-axis is the reward. You can check the rewards as the Episode progresses. If learning goes well, reward will increase as the episode progresses.

The horizontal axis is the episode, and the vertical axis is the reward, which is profit. The shaded area is the 95% confidence interval for the reward. Because 32 environments are trained in parallel and rewards are aggregated for each episode, there is a difference in the rewards aggregated for each episode in the 32 environments. Therefore, this was represented as a 95% confidence interval and shaded, and the average value for each episode of these environments was shown as a solid line. As the episode progressed, the reward increased, confirming that the training went well. The horizontally progressing part of the slope 0 without the last shade is because the maximum profit that can be obtained from the two models is limited. Since the maximum profit was reached, it was thought that overfitting would occur if the total number of steps was increased and learned. As a result of actually increasing the step to 1.25 million, it was confirmed that overfitting occurred and the performance deteriorated, so the number of steps was set to 1 million.

B. PERFORMANCE COMPARISON AND ANALYSIS OF THE TWO ROBOTS ON THE TEST DATA

In the previous section, it was confirmed whether the robot learned well with respect to the training data. Therefore, in this section, we checked how much the robot performed on the test data.

Figure 32 shows the investment made by the model trained on the test data. The x-axis is the episode, and the y-axis is the reward. You can check the rewards as the Episode progresses. If the performance is good, the average of the overall reward will be high and the variance will be low.

The trained model was run through 100 episodes on the test data. It was confirmed that there were many episodes in which the red model received a higher reward than the blue model. As a result of calculating the average reward for all episodes, it was confirmed that the red model performed better than the blue model trained using only APPL stock data with an average profit of -0.26 dollars.

In order to derive reliable results, the experiment in which the trained model proceeds with 100 episodes on the test data was repeated a total of 1000 times. And outliers were removed using the Inter Quartile Range (IQR). Figure 33 shows a histogram for this. It is a frequency distribution graph where the x-axis is reward (profit) and the y-axis is frequency. Through this figure, it is possible to check the average reward of each experiment for 1000 experiments. The better the performance, the more likely you will see an aspect that is skewed to the right side of the picture.

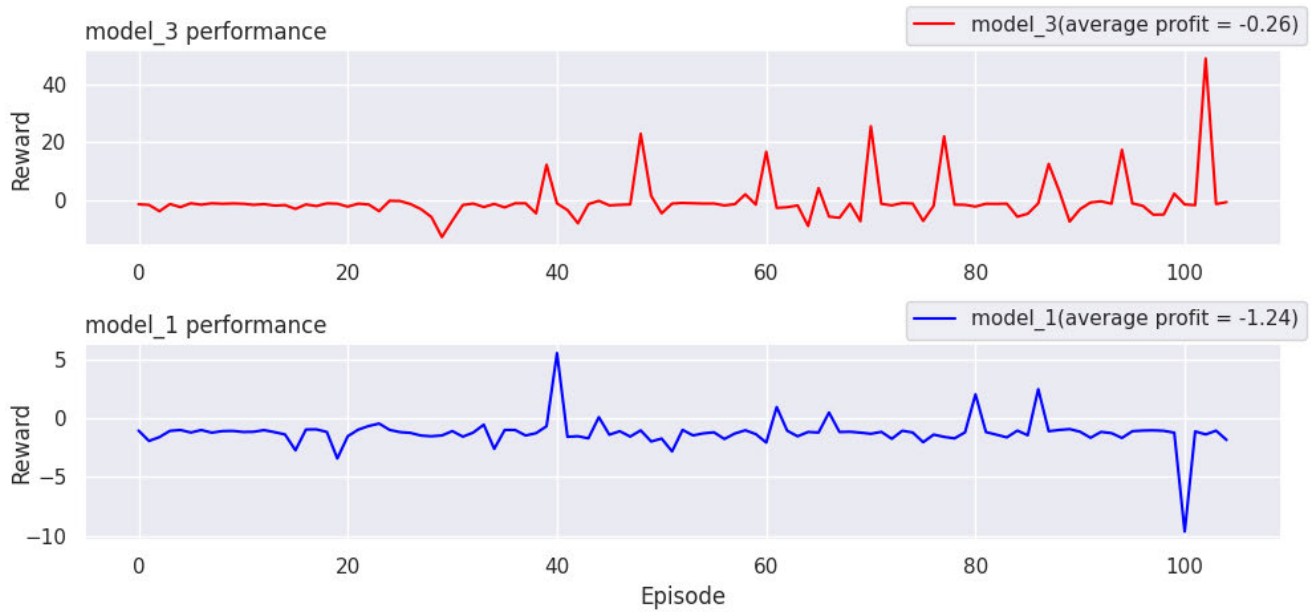


FIGURE 32. Performance on test data of Model 1 and Model 3.

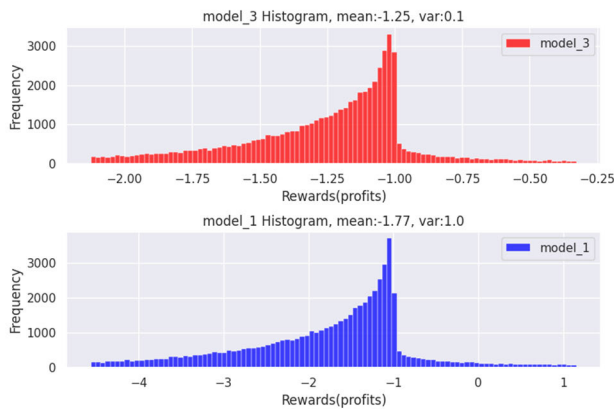


FIGURE 33. Histogram of the test performance of the two models over 1000 experiments.

Also, the more stable the reward is obtained, the smaller the variance will be.

As a result, the red model (Model 3) obtained an average reward of -1.25 and a variance of 0.1 , and the blue model (Model 1) obtained an average reward of -1.77 and a variance of 1 . Therefore, it can be judged that Model 3 is a better trained model than Model 1 in terms of stability and performance because the average reward is relatively high and the variance is significantly low.

Figure 34 shows the hypotheses tested through the results of the experiment just done. One-way ANOVA confirmed whether there was a difference in the performance of the two models at a confidence level of 95%.

And through a two-sample t-test, which model performed better was confirmed at a confidence level of 95%.

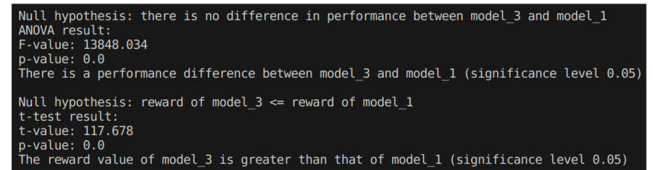


FIGURE 34. Hypothesis test of performance on test data of Model 1 and Model 3.

As a result of the first one-way ANOVA, the p value was 0 at the significance level of 0.05, and the null hypothesis was rejected, confirming that there was a difference in performance. In addition, through the two sample t-test, the null hypothesis was rejected with a p-value of 0 at a significance level of 0.05, confirming that the reward of the red model was greater.

V. CONCLUSION AND FUTURE WORK

Through the performance analysis of stock investment robots using deep reinforcement learning, we compared two models using single stocks and multiple stocks as learning data. As a result of the experiment, Model 3, a deep reinforcement learning model trained using multiple stocks, showed superior results in terms of investment performance and stability compared to Model 1 using a single stock. In the results, the variance of rewards was used as a measure of stability. This is because if a trading robot generates consistent profits over time, the variance will be small, and if the variance in profits increases, the variance will increase. From a stock investor's perspective, it is difficult to trust investment strategies with high volatility, so an investment robot with less volatility (less dispersion of rewards) was judged to be superior in

terms of stability. Additionally, the average return during the simulated investment period was used as a measure of performance.

Model 3 used Apple Inc, S&P 500 index, and US interest rates as training data. In a simulated investment experiment on test data from Apple Inc, the model achieved an average return of -\$1.25 and a variance of 0.1. Model 1 used Apple Inc data as training data. As a result of a simulated investment experiment on the same test data as Model 3, this model achieved an average return of -\$1.77 and a variance of 1.

In summary, Model 3, a deep reinforcement learning model that performed multi-item learning, showed better results (in terms of profit generation and stability) than Model 1, which performed single item learning. In other words, the simulated investment performance of Model 3 (average -\$1.25) was higher than the simulated investment performance of Model 1 (average -\$1.77). In addition, the simulated investment stability of Model 3 (variance 0.1) was better than that of Model 1 (variance 1).

In this paper, we show that the use of multiple-item learning can be excellent in developing a stock investment robot based on deep reinforcement learning. This is expected to be of great help in future related research and actual market application. These research results will contribute to the development and application of deep reinforcement learning models in the field of stock investment.

However, this paper needs some improvements in the future. A comparative study is needed that adds value-based algorithms in addition to the policy-based reinforcement learning algorithm we used. In addition, we would like to study ways to expect higher performance by adding big data, including various news, social media, and analyst reports, to the deep reinforcement learning model presented in this paper.

AVAILABILITY OF DATA AND MATERIALS

Ji-Heon Park, Jae-Hwan Kim, Jun-Ho Huh [50]:

“Source files of the dataset used: Google Drive,”

https://drive.google.com/drive/folders/1ZhVvr3_6u2Qv2HwIAarxiKQRDncks-Dl?usp=sharing

Ji-Heon Park, Jae-Hwan Kim, Jun-Ho Huh [55]:

“Source files of the dataset used: Github,”

<https://github.com/JHJP/Source-files-of-the-dataset-used>

REFERENCES

- [1] B. Graham and D. L. Dodd, *Security Analysis: Principles and Technique*. New York, NY, USA: McGraw-Hill, 1934.
- [2] J. J. Murphy, *Technical Analysis of the Financial Markets: A Comprehensive Guide To Trading Methods and Applications*. Baltimore, MD, USA: Penguin, 1999.
- [3] E. P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*. Hoboken, NJ, USA: Wiley, 2021.
- [4] J. D. Schwager, *A Complete Guide to the Futures Market: Technical Analysis, Trading Systems, Fundamental Analysis, Options, Spreads, and Trading Principles*. Hoboken, NJ, USA: Wiley, 2017.
- [5] E. W. Saad, D. V. Prokhorov, and D. C. Wunsch, “Comparative study of stock trend prediction using time delay, recurrent and probabilistic neural networks,” *IEEE Trans. Neural Netw.*, vol. 9, no. 6, pp. 1456–1470, Nov. 1998.
- [6] H. Rak Beom and H. Suck Cho, “A sensor-based navigation for a mobile robot using fuzzy logic and reinforcement learning,” *IEEE Trans. Syst., Man, Cybern.*, vol. 25, no. 3, pp. 464–477, Mar. 1995.
- [7] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [8] H. Kim, M. Jordan, S. Sastry, and A. Ng, “Autonomous helicopter flight via reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 16. Citeseer, 2003.
- [9] Y.-D. Song, Q. Song, and W.-C. Cai, “Fault-tolerant adaptive control of high-speed trains under traction/braking failures: A virtual parameter-based approach,” *IEEE Trans. Intell. Transp. Syst.*, vol. 15, no. 2, pp. 737–748, Apr. 2014.
- [10] J. M. Poterba and Lawrence H. Summers, “Mean reversion in stock prices: Evidence and implications,” *J. financial Econ.*, vol. 22, no. 1, pp. 27–59, 1988.
- [11] H. Buehler, L. Gonon, J. Teichmann, B. Wood, B. Mohan, and J. Kochems, “Deep hedging: Hedging derivatives under generic market frictions using reinforcement learning,” *Swiss Finance Inst., Res. Paper* 19-80, 2020.
- [12] J. C. Hull, *Options Futures and Other Derivatives*. Uttar Pradesh, India: Pearson Education, 2003.
- [13] X.-Y. Liu, Z. Xiong, S. Zhong, H. Yang, and A. Walid, “Practical deep reinforcement learning approach for stock trading,” 2018, pp. 1–7, *arXiv:1811.07522*.
- [14] Z. Zhang, S. Zohren, and R. Stephen, “Deep reinforcement learning for trading,” *J. Financial Data Sci.*, vol. 6, no. 1, 2020.
- [15] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, “Deep reinforcement learning for automated stock trading: An ensemble strategy,” in *Proc. 1st ACM Int. Conf. AI Finance*, 2020, pp. 1–8.
- [16] Q.-V. Dang, “Reinforcement learning in stock trading,” in *Proc. Int. Conf. Comput. Sci., Appl. Math. Appl.*, Cham, Switzerland: Springer, 2019, pp. 311–322.
- [17] L. Chen and Q. Gao, “Application of deep reinforcement learning on automated stock trading,” in *Proc. IEEE 10th Int. Conf. Softw. Eng. Service Sci. (ICSESS)*, Oct. 2019, pp. 29–33.
- [18] J. Li, R. Rao, and J. Shi, “Learning to trade with deep actor critic methods,” in *Proc. 11th Int. Symp. Comput. Intell. Design (ISCID)*, Dec. 2018, pp. 66–71.
- [19] S. D. Bekiros, “Fuzzy adaptive decision-making for boundedly rational traders in speculative stock markets,” *Eur. J. Oper. Res.*, vol. 202, no. 1, pp. 285–293, Apr. 2010.
- [20] Y. Zhang and X. Yang, “Online portfolio selection strategy based on combining experts’ advice,” *Comput. Econ.*, vol. 50, no. 1, pp. 141–159, Jun. 2017.
- [21] Y. Kim, W. Ahn, K. J. Oh, and D. Enke, “An intelligent hybrid trading system for discovering trading rules for the futures market using rough sets and genetic algorithms,” *Appl. Soft Comput.*, vol. 55, pp. 127–140, Jun. 2017.
- [22] Z. Jiang, D. Xu, and J. Liang, “A deep reinforcement learning framework for the financial portfolio management problem,” 2017, *arXiv:1706.10059*.
- [23] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [24] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Trans. Neural Netw.*, vol. 12, no. 4, pp. 875–889, Jul. 2001.
- [25] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, “Deep direct reinforcement learning for financial signal representation and trading,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 3, pp. 653–664, Mar. 2017.
- [26] X. Li, Y. Li, Y. Zhan, and X.-Y. Liu, “Optimistic bull or pessimistic bear: Adaptive deep reinforcement learning for stock portfolio allocation,” 2019, *arXiv:1907.01503*.
- [27] W. Bao and X.-Y. Liu, “Multi-agent deep reinforcement learning for liquidation strategy analysis,” 2019, *arXiv:1906.11046*.
- [28] D. Silver. (2015). *UCL Course on RL*. [Online]. Available: <https://www.davidsilver.uk/teaching/>
- [29] H. Dong, Z. Ding, and S. Zhang, *Deep Reinforcement Learning*. Singapore: Springer, 2020.

- [30] Q. Huang, "Model-based or model-free, a review of approaches in reinforcement learning," in *Proc. Int. Conf. Comput. Data Sci. (CDS)*, Aug. 2020, pp. 219–221.
- [31] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI gym," 2016, *arXiv:1606.01540*.
- [32] Y. Duan, X. Chen, R. Houthoof, J. Schulman, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1329–1338.
- [33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [34] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [35] P. Treleaven, M. Galas, and V. Lalchand, "Algorithmic trading review," *Commun. ACM*, vol. 56, no. 11, pp. 76–85, 2013.
- [36] G. E. Box and D. A. Pierce, "Distribution of residual autocorrelations in autoregressive-integrated moving average time series models," *J. Amer. Statist. Assoc.*, vol. 65, no. 332, pp. 1509–1526, 1970.
- [37] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *J. Econometrics*, vol. 31, no. 3, pp. 307–327, Apr. 1986.
- [38] O. Y. Grachev, "Application of time series models (ARIMA, GARCH, and ARMA-GARCH) for stock market forecasting," M.S. thesis, Northern Illinois Univ., 2017.
- [39] S. K. Sahu, A. Mokhadde, and N. D. Bokde, "An overview of machine learning, deep learning, and reinforcement learning-based techniques in quantitative finance: Recent progress and challenges," *Appl. Sci.*, vol. 13, no. 3, p. 1956, Feb. 2023.
- [40] T.-V. Pricope, "Deep reinforcement learning in quantitative algorithmic trading: A review," 2021, *arXiv:2106.00123*.
- [41] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Math. Finance*, vol. 33, no. 3, pp. 437–503, Jul. 2023.
- [42] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *Proc. AAAI Fall Symp.*, 2015.
- [43] R. H. Khan, J. Miah, M. M. Rahman, M. M. Hasan, and M. Mamun, "A study of forecasting stocks price by using deep reinforcement learning," in *Proc. IEEE World AI IoT Congr. (AIoT)*, Jun. 2023, pp. 250–255.
- [44] Y. Deng, Y. Kong, F. Bao, and Q. Dai, "Sparse coding-inspired optimal trading system for HFT industry," *IEEE Trans. Ind. Informat.*, vol. 11, no. 2, pp. 467–475, Apr. 2015.
- [45] G. J. Klir and T. A. Folger, *Fuzzy Sets, Uncertainty, and Information*. Upper Saddle River, NJ, USA: Prentice-Hall, 1987.
- [46] N. R. Pal and J. C. Bezdek, "Measuring fuzzy uncertainty," *IEEE Trans. Fuzzy Syst.*, vol. 2, no. 2, pp. 107–118, May 1994.
- [47] C.-T. Lin and C. S. G. Lee, "Neural-network-based fuzzy logic control and decision system," *IEEE Trans. Comput.*, vol. 40, no. 12, pp. 1320–1336, Dec. 1991.
- [48] D. Kwak, S. Choi, and W. Chang, "Self-attention based deep direct recurrent reinforcement learning with hybrid loss for trading signal generation," *Inf. Sci.*, vol. 623, pp. 592–606, Apr. 2023.
- [49] N. Majidi, M. Shamsi, and F. Marvasti, "Algorithmic trading using continuous action space deep reinforcement learning," *Expert Syst. Appl.*, vol. 235, Jan. 2024, Art. no. 121245.
- [50] J.-H. Park, J.-H. Kim, and J.-H. Huh, *Source Files of the Dataset Used: Google Drive*. [Online]. Available: https://drive.google.com/drive/folders/1ZhVvr3_6u2Qv2HwIAarxiKQRDncks-DI?usp=sharing
- [51] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 1, pp. 12348–12355, 2021.
- [52] J. Zhang, T. He, S. Sra, and A. Jadbabaie, "Why gradient clipping accelerates training: A theoretical justification for adaptivity," 2019, *arXiv:1905.11881*.
- [53] S. Gu, T. Lillicrap, I. Sutskever, and S. Levine, "Continuous deep Q-learning with model-based acceleration," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 2829–2838.
- [54] S. Sharma, S. Sharma, and A. Athaiya, "Activation functions in neural networks," *Towards Data Sci.*, vol. 6, no. 12, pp. 310–316, 2017.
- [55] J.-H. Park, J.-H. Kim, and J.-H. Huh, *Source Files of the Dataset Used: Github*. [Online]. Available: <https://github.com/JHJP/Source-files-of-the-dataset-used>



2017 to May 2019. He was an Undergraduate Researcher with the Data Science Laboratory, Korea Maritime and Ocean University, from March 2020 to August 2023.

JI-HEON PARK received the B.S. degree (Hons.) in data science from the Department of Data Informatics, (National) Korea Maritime and Ocean University, Republic of Korea, in August 2023. He is currently pursuing the master's degree with the Department of Business Administration, Graduate School of Business, Seoul National University, Seoul, Republic of Korea.

He was a Rifleman with the 52nd Infantry Division, Republic of Korea Army, from September



from September 1997 to August 1998. He was a Visiting Scholar of industrial and systems engineering with the Samuel Ginn College of Engineering, Auburn University, USA, from September 2011 to August 2012. He has been a Full Professor (Tenured) with the Department of Data Science, (National) Korea Maritime and Ocean University, Republic of Korea, since September 1991.

Dr. Kim delivered a keynote speech on "A Hybrid Metaheuristic for the Series-Parallel Redundancy Allocation Problem" at the 2008 KMU-HCMUT Joint Workshop and also gave tutorial lectures at the 2016 Korean Data and Information Science Society Fall Conference. Additionally, he has received numerous outstanding paper awards. After being published in the IEEE TRANSACTIONS ON RELIABILITY, in 1993, he has served as a Reviewer for IEEE TRANSACTIONS for over 30 years.

JAЕ-HWAN KIM received the B.S. degree in industrial engineering from Korea University, Seoul, Republic of Korea, in February 1984, and the M.S. and Ph.D. degrees from the Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology (KAIST), Republic of Korea, in February 1987 and February 1993, respectively.

He was a Postdoctoral Researcher with the University of Illinois, USA, (Supported by KOSEF),

from September 1997 to August 1998. He was a Visiting Scholar of industrial and systems engineering with the Samuel Ginn College of Engineering, Auburn University, USA, from September 2011 to August 2012. He has been a Full Professor (Tenured) with the Department of Data Science, (National) Korea Maritime and Ocean University, Republic of Korea, since September 1991.

Dr. Kim delivered a keynote speech on "A Hybrid Metaheuristic for the Series-Parallel Redundancy Allocation Problem" at the 2008 KMU-HCMUT Joint Workshop and also gave tutorial lectures at the 2016 Korean Data and Information Science Society Fall Conference. Additionally, he has received numerous outstanding paper awards. After being published in the IEEE TRANSACTIONS ON RELIABILITY, in 1993, he has served as a Reviewer for IEEE TRANSACTIONS for over 30 years.



neering from the Department of Computer Engineering, Graduate School, Pukyong National University, in February 2016.

He finished the Cooperative Marine Science and Engineering Program, Texas A&M University at Galveston, USA, in August 2006. He was a Research Professor with Dankook University, Jukjeon, Yongin, Republic of Korea, from July 2016 to September 2016. He was an Assistant Professor with the Department of Software, Catholic University of Pusan, Republic of Korea, from December 2016 to August 2019. He was an Assistant Professor with the Department of Data Informatics, (National) Korea Maritime and Ocean University, Busan, Republic of Korea, from September 2019 to September 2021. Since October 2021, he has been an Associate Professor (Tenured) with the Department of Data Informatics/Data Science, (National) Korea Maritime and Ocean University. Since September 2020, he has been

JUN-HO HUH (Member, IEEE) received the B.S. degree in science from the Department of Major of Applied Marine Sciences, the B.E. degree in engineering (double major) from the Department of Computer Engineering, Jeju National University at Ara, Jeju, Republic of Korea, in August 2007, the M.A. degree in education from the Department of Computer Science Education, Pukyong National University, Daejeon, Busan, Republic of Korea, in August 2012, and the Ph.D. degree in engineering from the Department of Computer Engineering, Graduate School, Pukyong National University, in February 2016.

He finished the Cooperative Marine Science and Engineering Program, Texas A&M University at Galveston, USA, in August 2006. He was a Research Professor with Dankook University, Jukjeon, Yongin, Republic of Korea, from July 2016 to September 2016. He was an Assistant Professor with the Department of Software, Catholic University of Pusan, Republic of Korea, from December 2016 to August 2019. He was an Assistant Professor with the Department of Data Informatics, (National) Korea Maritime and Ocean University, Busan, Republic of Korea, from September 2019 to September 2021. Since October 2021, he has been an Associate Professor (Tenured) with the Department of Data Informatics/Data Science, (National) Korea Maritime and Ocean University. Since September 2020, he has been

the Center Chair (Director) of the Big Data Center for Total Lifecycle of Ship-building and Shipping, (National) Korea Maritime and Ocean University. Since September 2022, he has been a Joint Associate Professor (Tenured) with the Global R&E Program for Interdisciplinary Technologies of Ocean Renewable Energy (BK 21 Four Research Group), Interdisciplinary Major of Ocean Renewable Energy Engineering, (National) Korea Maritime and Ocean University. He was the General/Head Professor with the Catholic University of Pusan on International Game Exhibition G-Star 2017 (G-Star 2017). He is the author of the book *Smart Grid Test Bed Using OPNET and Power Line Communication* (USA: IGI Global, 2017, 425) and *Principles, Policies, and Applications of Kotlin Programming* (USA: IGI Global, 2023, 457). He has authored/edited ten books and edited ten special issues in reputed Clarivate Analytics Index journals. He has published more than 100 articles in the Clarivate Analytics Index (SCI/SCIE/SSCI indexed) with over 3200 citations and has an H-index of 32.

Dr. Huh has been a Technical Committee (TC) Member of the International Federation of Automatic Control (IFAC), CC 1 (Systems and Signals), TC 1.5. (Networked Systems); IFAC, Computers, Cognition and Communication (CC 3), TC 3.2 (Computational Intelligence in Control); and IFAC, CC 7 (Transportation and Vehicle Systems), TC 7.2. (Marine Systems), since 2017. Since 2020, he has been a TC Member of IFAC, CC 2 (Design Methods), TC 2.6. (Distributed Parameter Systems). He received the Best Paper Minister Award (Ministry of Trade, Industry and Energy, Korean Government) from the 16th International Conference on Control, Automation, and Systems, in October 2016, ICROS with IEEE Xplore, the

Springer Nature Most Cited Paper Award, and the *Human-centric Computing and Information Sciences* Most Cited Paper Award, in 2019 (Research published in the journal, from 2016 to 2018; SCIE IF=6.558). He was awarded the Commendation for Meritorious Service in the Promotion of Busan's Data Industry, in December 2023 (Commendation by the Mayor of Busan). He was the Organizing Chair of the 15th International Conference on Multimedia Information Technology and Applications (MITA 2019: University of Economics and Law (UEL), Vietnam National University Ho Chi Minh City, Vietnam) and the 17th International Conference on Multimedia Information Technology and Applications (MITA 2021: Jeju KAL Hotel, ROK). He was the Managing Editor (ME) of the *Journal of Information Processing Systems* (JIPS), Korea Information Processing Society (SCOPUS/ESCI indexed), from January 2020 to December 2021, and the *Journal of Multimedia Information System* (JMIS), Korea Multimedia Society (EI/KCI indexed), from January 2017 to December 2022. Since July 2023, he has been an Editor-in-Chief (EiC) of *Global Convergence Research*, Global Convergence Research Academy, Pukyong National University. He is an Associate Editor (AE) of *Journal of Information Processing Systems* (JIPS), Korea Information Processing Society (SCOPUS/ESCI indexed); the *Journal of Multimedia Information System* (JMIS), Korea Multimedia Society (EI/KCI indexed); and *Human-centric Computing and Information Sciences* (HCIS), Springer Berlin Heidelberg (SCIE IF=6.558). For more information, see <https://scholar.google.co.kr/citations?user=cr5wjNYA-AAAJ&hl=en>

• • •